Documentation Technique & Specification API

Plateforme de Billetterie et Suivi de Flotte Interurbain — MaliTrans Backend

Projet : MaliTrans **Stack :** Laravel 11 / Sanctum / MySQL **Version :** 1.2 (Octobre 2026)

1. Vue d'Ensemble du Projet

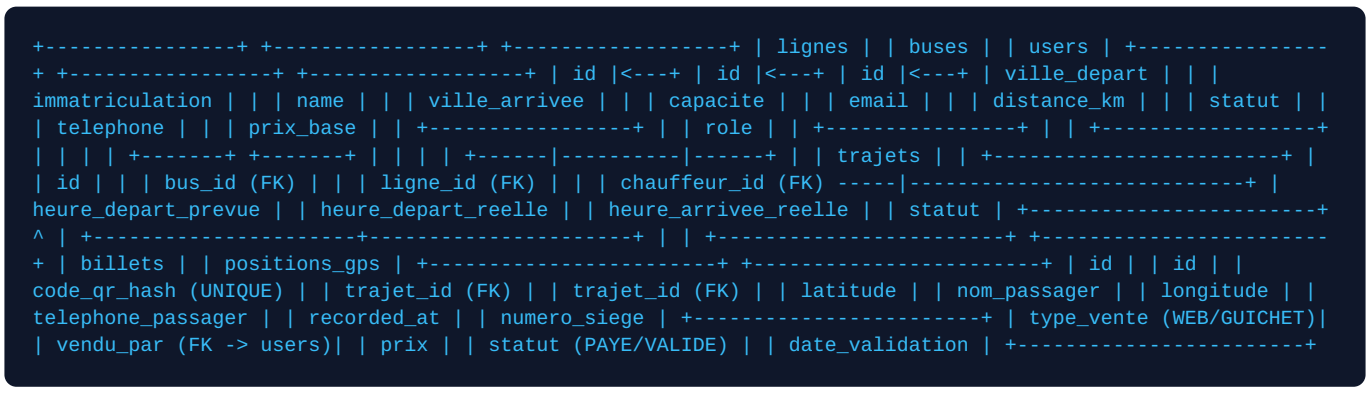
MaliTrans est une solution logicielle globale dédiée à la numérisation, la sécurisation et la gestion en temps réel des flux de transport routier interurbain au Mali. La plateforme interconnecte les acteurs clés de la chaîne de transport : les administrateurs de la compagnie, les guichetiers en station et les chauffeurs en ligne.

> **Architecture Tripartite**
> Le système s'articule autour de trois composants majeurs parfaitement synchronisés :

- **Portail Web d'Administration & Guichet (Laravel / Blade / REST) :** Gestion centrale du référentiel (flotte de bus, lignes interurbaines, affectations chauffeurs, grille tarifaire) et interface de vente de billets avec génération de QR code et impression thermique pour guichet.
- **Backend API REST (Laravel 11 & Sanctum) :** Moteur applicatif sécurisé assurant la logique métier critique (transactions atomiques de vente, vérification de capacité en temps réel, génération d'empreintes QR infalsifiables et contrôle d'accès par rôles).
- **Application Mobile Conducteur (Flutter) :** Outil embarqué dédié aux chauffeurs pour la consultation de la feuille de route active, la vérification instantanée des billets par scan QR code et la transmission continue de la position GPS du véhicule.

2. Architecture de la Base de Données

Le schéma relationnel est optimisé pour garantir l'intégrité référentielle, prévenir le sur-remplissage des véhicules et assurer une traçabilité complète des ventes et validations.



Détail des Rôles et Permissions (`users.role`)

Rôle	Périmètre d'Accès & Description	Routes Autorisées
`admin`	Gestion complète de la flotte (bus, lignes), création des comptes employés et planification des trajets.	`/api/v1/admin/*`
`guichetier`	Consultation du programme des départs, émission et impression des billets passagers au guichet.	`/api/v1/guichet/*`
`chauffeur`	Accès mobile : consultation du trajet attribué, validation des billets par scan QR et émission GPS.	`/api/v1/driver/*`

3. Logique Métier Centralisée : `TicketService`

Pour garantir une maintenance rigoureuse et éviter toute duplication de code, l'ensemble des règles métier relatives à la réservation, l'émission et la vérification des titres de transport est encapsulé dans la classe `App\Services\TicketService`.

A. Émission Atomique de Billet (`createTicket`)

```
public function createTicket(Trajet $trajet, array $data): Billet
{
    return DB::transaction(function () use ($trajet, $data) {
        // 1. Verrouillage et contrôle strict de la capacité
        $ticketsVendus = $trajet->billets()->where('statut', '!=', 'ANNULE')->count();
        if ($ticketsVendus >= $trajet->bus->capacite) {
            throw new Exception("Le bus est complet pour ce trajet.");
        }

        // 2. Génération de l'empreinte QR unique
        $qrHash = 'MALITRANS-' . strtoupper(Str::random(16)) . '-' . time();

        // 3. Création du billet
        return $trajet->billets()->create([
            'code_qr_hash' => $qrHash,
            'nom_passager' => $data['nom_passager'],
            'telephone_passager' => $data['telephone_passager'] ?? null,
            'numero_siege' => $data['numero_siege'] ?? null,
            'prix' => $data['prix'] ?? $trajet->prix_base,
            'vendu_par' => auth()->id(),
            'type_vente' => 'GUICHET',
            'statut' => 'PAYE',
        ]);
    });
}
```

B. Validation par QR Code (`validateTicketByQrCode`)

```
public function validateTicketByQrCode(string $qrHash, int $trajetId): Billet
{
    $billet = Billet::where('code_qr_hash', $qrHash)->firstOrFail();

    if ($billet->trajet_id !== $trajetId) {
        throw new ValidationException("Ce billet n'est pas valable pour ce trajet.");
    }

    if ($billet->statut === 'VALIDE') {
        throw new ValidationException("Ce billet a déjà été utilisé.");
    }

    if ($billet->statut === 'ANNULE') {
        throw new ValidationException("Ce billet a été annulé.");
    }

    $billet->update([
        'statut' => 'VALIDE',
        'date_validation' => now(),
    ]);

    return $billet;
}
```

4. Spécification Générale des Endpoints API

Toutes les requêtes d'API nécessitent l'utilisation du protocole HTTPS et l'envoi des entêtes HTTP d'authentification ci-dessous :

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer <SANCTUM_TOKEN>
```

A. Authentification (`/api/v1/auth`)

POST Connexion Utilisateur

`/api/v1/auth/login`

Body Request :

```
{
  "email": "admin@malitrans.ml",
  "password": "AdminSecurePassword2026!"
}
```

Réponse Succès (200 OK) :

```
{
  "success": true,
  "token": "1|qX89K2P1L4M6Q9Z3B...",
  "user": { "id": 1, "name": "Super Admin", "role": "admin" }
}
```

B. Espace Administration (`/api/v1/admin`)

POST Créer un Nouvel Employé

`/api/v1/admin/users`

```
// Body
{
  "name": "Moussa Guichet",
  "email": "guichet@malitrans.ml",
  "telephone": "+22370000001",
  "password": "password123",
  "role": "guichetier"
}
```

POST Enregistrer un Bus dans la Flotte

`/api/v1/admin/buses`

```
// Body
{
  "immatriculation": "ML-555-MD",
  "capacite": 45,
  "statut": "EN_SERVICE"
}
```

POST Créer une Ligne Interurbaine

`/api/v1/admin/lignes`

```
// Body
{
  "ville_depart": "Bamako",
  "ville_arrivee": "Kayes",
  "prix_base": 10000,
  "distance_km": 500
}
```

POST Planifier un Trajet

`/api/v1/admin/trajets`

```
// Body
{
  "ligne_id": 1,
  "bus_id": 1,
  "chauffeur_id": 3,
  "heure_depart_prevue": "2026-11-20 09:00:00",
  "prix_base": 10000
}
```

C. Espace Guichet (`/api/v1/guichet`)

GET Lister les Trajets Disponibles

`/api/v1/guichet/trajets`

```
// Response (200 OK)
{
  "success": true,
  "data": [
    {
      "id": 1,
      "heure_depart_prevue": "2026-11-20 09:00:00",
      "statut": "PLANIFIE",
      "bus": { "immatriculation": "ML-555-MD", "capacite": 45 },
      "ligne": { "ville_depart": "Bamako", "ville_arrivee": "Kayes" }
    }
  ]
}
```

POST Émettre un Billet Passager

`/api/v1/guichet/trajets/{trajet}/billets`

```
// Body
{
  "nom_passager": "Ibrahim Bassoum",
  "telephone_passager": "+22370000000",
  "numero_siege": 12
}

// Response (201 Created)
{
  "success": true,
  "message": "Billet émis avec succès.",
  "data": {
    "id": 101,
    "code_qr_hash": "MALITRANS-A8F9K2L1M4P6Q9Z3-1730245200",
    "nom_passager": "Ibrahim Bassoum",
    "statut": "PAYE"
  }
}
```

D. Espace Chauffeur / Mobile Flutter (`/api/v1/driver`)

GET Obtenir le Trajet Actif du Chauffeur

`/api/v1/driver/trajet-actif`

POST Valider un Billet par QR Code

`/api/v1/driver/trajets/{trajet}/valider-billet`

```
// Body
{
  "code_qr_hash": "MALITRANS-A8F9K2L1M4P6Q9Z3-1730245200"
}

// Response (200 OK)
{
  "success": true,
  "message": "Billet valide. Bon voyage !"
}

// Response Erreur (422 Unprocessable Entity - Déjà scanné)
{
  "success": false,
  "message": "Ce billet a déjà été utilisé."
}
```

POST **Mettre à Jour la Position GPS**

`/api/v1/driver/trajets/{trajet}/gps`

```
// Body
{
  "latitude": 12.639232,
  "longitude": -7.998234
}
```

5. Recommandations de Déploiement & Intégration

- **Impression Thermique Guichet :** L'interface web guichet doit intercepter le champ `code_qr_hash` de la réponse JSON d'émission et utiliser la librairie JavaScript `qrcode.js` pour générer dynamiquement le visuel du QR code à envoyer à l'imprimante thermique.
- **Scanner Mobile Flutter :** Intégrer la librairie `mobile_scanner` dans l'application chauffeur pour scanner l'empreinte texte imprimée sur le ticket et transmettre directement la chaîne brute à l'API de validation.
- **Résilience Réseau (Mode Déconnecté Chauffeur) :** Prévoir une synchronisation SQLite locale sur l'application mobile Flutter pour stocker temporairement les billets validés lors de la traversée de zones à faible couverture réseau sur les axes nationaux.